

Concordance

Every page of the tool, in plain words — and an honest list of everything it will not do.

PART 1 — WHAT EACH PAGE DOES

- 1. Overview — the front door
- 2. Dashboard — the report, explained
- 3. Dataset + SQL — everything at once
- 4. BRD & FRD — the documents
- 5. Browse objects — the model itself
- 6. Drift — what changed
- 7. Warehouse — does the database agree?
- 8. To confirm — the queue
- 9. Copilot — ask it questions
- 10. Command line — no browser needed

PART 2 — LIMITATIONS

- 1. Measures that cannot become SQL
- 2. Numbers and formatting
- 3. What cannot be read out of the file
- 4. Warehouse comparison
- 5. Charts and the dashboard
- 6. Chatbot
- 7. Verification
- 8. Scope

Each limitation is labelled as a deliberate refusal, something not built yet, or something the environment blocks.

TEAM	MetaRadar · Amity University
WHAT IT IS	Documentation and verification for Power BI semantic models
READS	.pbix files · .SemanticModel (TMDL) folders · SQL warehouses
PRODUCES	BRD & FRD documents · DAX → SQL translations · a browsable model · drift reports
VERIFIED BY	1,274 passing tests across 64 files

Part 1 — What each page does

Ten places you can be in this tool. Each one answers a different question.

Overview THE FRONT DOOR

Tells you what you have just opened: how many tables, measures and report pages the file holds, and what the tool could *not* read from it.

WHY IT EXISTS Two different people open this tool. A reviewer wants to know what the model reports on. Whoever maintains it wants the formulas. This page points each of them at the right tab by the question they are asking, not by the tab's name.

WHAT TO DO HERE Read the counts, glance at the blind spots, then pick a door.

Dashboard THE REPORT, EXPLAINED

Shows the tiles from the actual Power BI report, and under each one the formula that produces it — the DAX the author wrote, and the SQL that means the same thing.

WHY IT EXISTS Somebody asked twice in one review: "how do I know which formula belongs to which number on the dashboard?" Every other page starts from the model. This one starts from the screen you are looking at and works backwards.

WHAT YOU SEE Each headline number, computed by actually running its query against the file's own rows — not copied from the report. Charts break it down by category, over time, and on a map where the file records coordinates. Click a bar and the whole page re-runs filtered to it.

WORTH KNOWING Numbers are shown the way the file asks for them — \$45,184,554 , 42.29% — and the raw figure the query returned is always shown beside them, so you can check one against the other.

Dataset + SQL EVERYTHING AT ONCE

Every measure in the file, its DAX, and the SQL query that would produce the same number — all on one page.

WHY IT EXISTS People arrive asking "what does this dataset compute, and how would I get the same numbers myself?" Answering that used to mean opening each measure one at a time and copying it out.

THE GRAIN SELECTOR At the top you choose "one row per what" — per month, per store, per product. This is not a filter. A DAX measure has no single SQL translation until you say what you are grouping by, because **GROUP BY** is the filter context written down. Change it and every query on the page is rewritten.

ALSO HERE The joins between tables, written as SQL, in five dialects: DuckDB, Snowflake, Databricks, Redshift and Athena.

BRD & FRD THE DOCUMENTS

The Business and Functional Requirements Documents, written from the model itself. Downloadable as Word or Markdown.

WHY IT EXISTS	This is the thing the brief asked for: automating BRD and FRD creation from a Power BI file.
WHAT MAKES IT TRUSTWORTHY	Every sentence is tied to the object it came from and carries a fingerprint of that object. Every sentence is also marked with how it was arrived at: <i>the model states this directly</i> , <i>inferred from how the model is built</i> , or <i>the model could not settle this — somebody has to decide</i> .
THE BUSINESS VERSION	Written for someone who does not read DAX. Each metric says what it currently comes to on the file's own data, so you are approving a definition you have seen produce a number.

Browse objects THE MODEL ITSELF

The model as a tree: tables own their columns, measures and drill-down paths. Click any object to see its definition, what it depends on, and what breaks if you change it.

WHY A TREE	A semantic model is mostly hierarchical, and a tree renders that at any size without zooming, dragging or untangling lines. The genuinely graph-shaped question — "what breaks if I change this" — is answered by naming the affected objects instead.
WHAT TO DO HERE	Impact analysis before you edit something.

Drift WHAT CHANGED

Compares two versions of the same model and tells you what moved — and, more usefully, which written requirements are now in question because of it.

WHY IT EXISTS	Any diff can list changed objects. The point here is the second half: naming the requirements whose evidence sits on those objects, so a signed document can be checked rather than trusted.
WHAT YOU SEE	Each change with the fingerprint on both sides and the text each was taken from — because "the hash moved" is not something a reviewer can act on.

Warehouse DOES THE DATABASE AGREE?

Takes a Power BI measure and a warehouse view that claims to compute the same thing, and shows whether they actually read the same tables, columns and aggregations.

THREE VERDICTS, NOT TWO	<i>Agrees</i> , <i>differs</i> , and <i>needs review</i> . The third is not hedging: deciding whether two arbitrary expressions in two languages produce the same number is not decidable in general, so a difference that may or may not matter is reported as exactly that.
WHAT YOU SEE	The two definitions side by side with the differing parts marked. The verdict is just a summary of that.

To confirm THE QUEUE

Every statement the tool inferred rather than read, in one place, waiting for a person to agree or disagree.

WHY IT EXISTS The model does not say what an inactive relationship is *for*. A sentence claiming to know is a guess, and guesses belong in front of a person before a document goes anywhere.

WHAT TO DO HERE Accept, reject or annotate each one. Decisions are recorded and flow back into the documents.

Copilot SIDE PANEL, ALWAYS THERE

Ask questions about the model in plain English — "what feeds Net Sales?", "which measures use the Calendar table?" — and get answers.

HOW IT ANSWERS Not from memory. It calls tools that read the extracted model, so an answer about your file comes from your file.

NEEDS An API key in the server's environment. Without one it says so plainly rather than making something up.

Command line NO BROWSER NEEDED

The same work, scriptable — for CI, for a batch of files, or for a machine with no screen.

COMMANDS `extract · inspect · explain · verify · ask · document · snapshot · drift · auditpack · serve`

MOST USED `document --type brd` writes the business document; `auditpack` bundles the model, its documents and the evidence into one reviewable archive.

Part 2 — Limitations

What this tool does not do, and why. Three kinds are mixed together below and each is labelled: a deliberate refusal, something not built yet, and something blocked by the environment rather than by the design.

The general rule. Where this tool cannot answer something, it says so in the place the answer would have gone. It does not fill a gap with a plausible guess, and it does not quietly drop the question. Almost every limitation below is visible somewhere in the interface or in the generated document.

1 Measures that cannot become SQL

The largest and most visible limit, and the one most often mistaken for a bug.

22 DAX functions stop a translation BY DESIGN

A measure using any of them produces no SQL. Instead the tool explains, in report terms, why no single query can stand for it.

- **Depends on what the user clicked** — ALL , ALLSELECTED , ALLEXCEPT , REMOVEFILTERS , ISINSCOPE , SELECTEDVALUE , ISFILTERED , HASONEVALUE , RANKX , TOPN
- **Depends on which period the report is showing** — TOTALYTD , TOTALQTD , TOTALMTD , DATESYTD , DATESBETWEEN , DATESINPERIOD , DATEADD , SAMEPERIODLASTYEAR , PARALLELPERIOD
- **Changes the joins while it runs** — USERRELATIONSHIP , CROSSFILTER
- **Points at a row being worked through elsewhere** — EARLIER

Why: these measures genuinely have no fixed answer. Their result changes with what is on screen. Producing a query anyway would give a number that looks authoritative and is wrong. Eight time-shift patterns that are unambiguous — such as previous month — were unblocked and now translate.

What this costs in practice BY DESIGN

MODEL	MEASURES	FIGURES COMPUTED	MAIN REASON FOR THE REST
Sales & Returns	58	6	17 use ALL ; 12 are per-period only; 7 use ISINSCOPE
Store Sales	32	29	1 uses ALL ; 1 is a constant; 1 ambiguous column name
Supply Chain	4	4	—

Every refusal names a construct. Across all seven sample models there is no measure left that fails for an unstated reason — the last one, Supply Chain's `Manufactured (%)` , turned out to be a plain nested IF the parser could not read because of two ampersands in the middle of it.

Read this carefully: 6 of 58 looks like a failure and is not. Microsoft's Sales & Returns sample is a demonstration of interactive DAX — it is built almost entirely from the functions above. A model written for reporting rather than for demonstration behaves like Store Sales.

2 Numbers and formatting

Three kinds of format string are not applied **BY DESIGN**

Date pictures (`dd-mm-yyyy` , `Long Date`), the trailing comma that divides by a thousand, conditional formats with more than three sections, and formats that hide negatives. Nine of the twelve distinct formats across the sample files render; the three declined are all date pictures.

Why: *a half-applied format changes the number rather than its appearance. Where a format is declined the figure is shown exactly as the query returned it — always true, even when it is not what Power BI would draw.*

Joins keep unmatched rows, unless the file says not to **BY DESIGN**

A Power BI relationship does not throw rows away: a sale whose product id matches no product is still counted, under a blank member. So a translation joins outward from the table being aggregated and keeps its rows, and any group with no value is labelled (`Blank`) rather than dropped. Where the author ticked "assume referential integrity", the file has said those rows cannot exist, and the join is an inner one to match.

Why it is worth a paragraph: *the alternative is not a missing row, it is a wrong explanation. With one orphaned sale worth 700 of 1,000, an inner join reported the total as 1,000, the parts as summing to 300, and then — because the parts did not reach the whole — announced the measure as an average or a ratio. A statement about the tool's own dropped row, presented as a fact about somebody's measure.*

Figures come from a snapshot, and the document says when **BY DESIGN**

Every number is computed by running the measure's own SQL against the rows stored inside the `.pbix` file. That is what makes it checkable. It is also, by definition, as old as the last refresh — so the documents now carry a **Data loaded** line beside the Power BI version that wrote the file. Sales & Returns states figures from December 2019 in a document generated today, which is worth knowing before signing it.

When the file says nothing, *the line is absent rather than guessed. Five of Store Sales' tables have never recorded a refresh at all.*

A DirectQuery table has no rows here **BY DESIGN**

Its data is fetched from the source when the report runs, so the file carries none. The tool reports those tables by name and explains that the absence is the file's shape rather than a limit of the tool.

A TMDL folder produces no figures at all **BY DESIGN**

A `.SemanticModel` folder describes the model without carrying the data behind it. Everything else works — documents, browsing, drift, the chatbot — but nothing on those pages can state what a measure comes to.

3 What cannot be read out of the file

Translated object names **NOT BUILT**

A model translated into another language carries those names, but each is keyed by a bare number whose meaning depends on the object type, and the library used to read the file publishes no mapping for it.

Why not guessed: *attaching one measure's French name to a different measure is worse than saying the translations were not read. The count is reported.*

Some row-level security detail **READER LIMIT**

Table filters inside a security role are read in full. A role that filters no table at all produces no row in the source, and each role's model-level permission is never exposed — so a role appears only if it restricts at least one table. Stated wherever security is reported.

Q&A synonyms — read, counted, deliberately unused BY DESIGN

Power BI stores the alternative words people might use for a table or column. This looked like the best remaining source for a business glossary. Reading them settled it: across the three sample files all 565 terms are machine-generated — either the object's own name with the spaces put back ("location ID" for `LocationID`) or a thesaurus suggestion. Power BI proposes "artifact advertised" for `Product Advertised`.

Why: *a glossary where a reader cannot tell which entries a person stood behind is worse than no glossary. The count is reported with this reasoning attached.*

Five internal sources, each with a recorded reason BY DESIGN

Storage statistics and per-column storage structures (the data itself answers the same questions), engine settings, internal annotations whose meaningful parts are read from better places, and calculated-table DAX which is read from partitions instead. On the TMDL side: source column names (identical to the model's name in all six sample models), compatibility level, and a KPI's choice of icon.

Kept honest: *both audits run as tests. A source that starts carrying data, or one the reader quietly stops reading, fails the build.*

Report layout, theming and interaction settings BY DESIGN

Where a tile sits on the canvas, its size, colours, fonts and the mobile layout are not read. Neither are drill-through page bindings or per-pair visual interaction overrides.

Why: *this documents what a report says, not how it is arranged. Drill-through bindings are a genuine gap — a page reached by drilling through always shows its figures under a filter passed in from elsewhere — and are the clearest next thing to build.*

Filters in shapes the tool does not recognise BY DESIGN

Report-, page- and visual-level filters are read from both report formats, including Top-N filters. Anything else is reported as *present but not interpreted*, by name.

Why it matters: *a filter nobody mentions is a filter nobody checks. This is the failure the whole area exists to prevent — Power BI showing 387K where the tool showed 1.2M, because the report was pinned to June and nothing said so.*

4 Warehouse comparison

Snowflake and Databricks are not verified against a live account ENVIRONMENT

The SQL path is exercised end to end against DuckDB, whose information schema follows the same standard Snowflake and Databricks implement. The Snowflake connector is unit-tested against a fake cursor, which proves the query text, the case handling and the wiring.

What that cannot prove: *a real handshake. This project's sandbox blocks outbound connections to `*.snowflakecomputing.com` at the network policy layer — confirmed against a real trial account, not assumed. Authenticating for real has to happen from a machine outside that policy.*

The comparison is structural, not semantic BY DESIGN

DAX and SQL never hash alike even when they compute the same number, so what is compared is what each definition reads — tables, columns, aggregations. Two definitions can differ structurally and produce identical numbers, or match structurally and differ in a `WHERE` clause.

Why "needs review" exists: *proving two arbitrary expressions equivalent is undecidable in general. Collapsing that verdict into either neighbour would misreport it.*

5 Charts and the dashboard

Fixed bounds on what gets drawn **BY DESIGN**

A chart shows at most 10 slices, with the remainder folded into a labelled "N more". A column is only offered as a dimension when it has between 2 and 30 distinct values. A time series is capped at 120 points, a sparkline at 40.

Why: *a chart with 400 bars is not a chart. The folded slice is labelled and counted rather than dropped.*

The map needs coordinates in the file **BY DESIGN**

Where the model declares latitude and longitude columns, points are plotted at them. Where it carries only a postcode or a city name, no map is drawn and the reason is stated.

Why: *turning a postcode into a position means a lookup against data outside the file, and a map drawn from a guess puts a location somewhere it is not. Google Maps as a base layer needs a key in the environment; without one, a built-in tile map is used and the points are identical either way.*

6 Chatbot

Needs a working API key **DEPLOYMENT**

Five providers are supported with automatic fallback between them. With no key configured the panel says so rather than degrading into guesswork.

It can only answer what the tools can read **BY DESIGN**

Questions are answered by calling functions over the extracted model. A question the tools cannot reach gets "I cannot answer that from this model" rather than a plausible fabrication.

7 Verification

1,274 tests pass; 44 are skipped **ENVIRONMENT**

43 of the 44 skip for one reason: `AdventureWorks_Sales.pbix` is not in the repository. It was only ever an upload and was lost when a container was reset. The tests that need it — 16 covering implicit measures, 7 covering hierarchies — are written and will run the moment the file is present. The 44th needs an optional dependency for one authentication mode.

What is not covered by tests **ENVIRONMENT**

A live Snowflake handshake, and a real API key against a real model provider. Both are blocked by the sandbox rather than unwritten.

8 Scope

Three kinds of input **BY DESIGN**

`.pbix` files (both the legacy report format and the newer per-file one), `.SemanticModel` TMDL folders, and SQL warehouses through their information schema. A model behind a Power BI Service URL can be opened by link, but that path is built and waiting for API credentials — there is no key configured for `app.powerbi.com`.